Final Report

The Automaton Auditor

Week 2 Challenge

Martha Ketsela Mengistu

February, 2026

1. Executive Summary

The Automaton Auditor is a deep, multi-agent governance system built on LangGraph that implements a "Digital Courtroom" architecture for autonomous code review and quality assurance. The system orchestrates a hierarchical swarm of specialized agents: three parallel Forensic Detectives gather objective evidence from a target repository and PDF report, three conflicting Judge personas (Prosecutor, Defense, Tech Lead) deliberate on the evidence through a dialectical process, and a deterministic Chief Justice synthesizes their opinions into a final, actionable audit report.

A self-audit using the system itself demonstrates significant progress. The final implementation scores 4.0/5.0, a substantial improvement from the initial 1.9/5.0 identified in earlier audits. This leap was driven by the MinMax feedback loop, where a peer agent's critique was instrumental in identifying and rectifying critical gaps directly leading to a more robust and compliant architecture. While the core engine is strong, particularly in executing the architectural diagram by the agent and making the agent work for different rubrics, remains an area for improvement.

2. Architecture

The Automaton Auditor is not a simple linear script but a hierarchical state machine. Its power comes from a deliberate separation of concerns, mirroring a legal system to ensure objective, nuanced, and auditable evaluations.

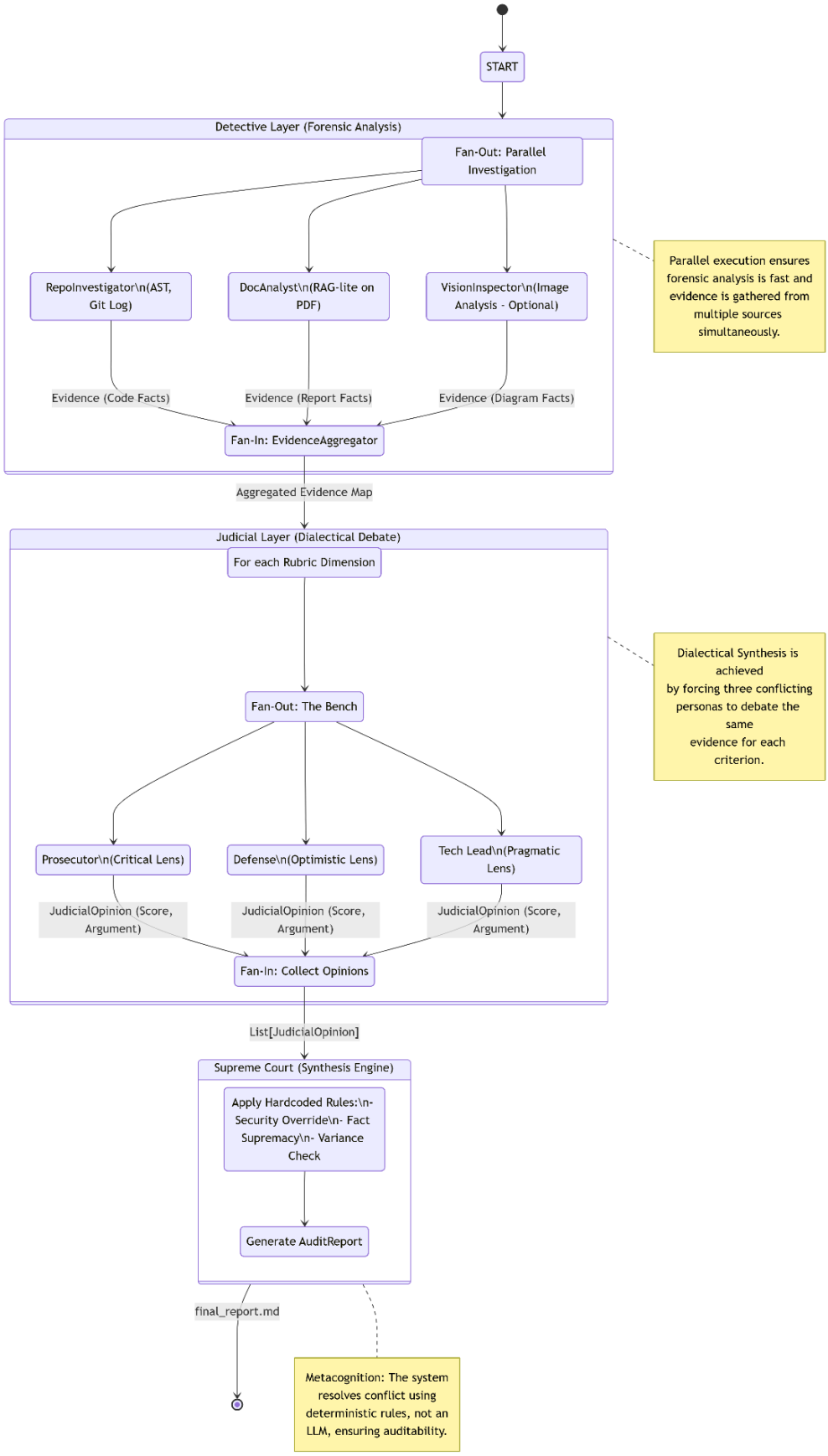
Core Orchestration: Fan-Out / Fan-In

The system's workflow is built on two primary parallel phases, a classic implementation of the fan-out/fan-in pattern. This design choice was driven by the need for both efficiency (parallel evidence gathering) and dialectical tension (parallel judgment).

1. **The Detective Layer (Fan-Out)**: From the single START node, the graph immediately fans out to three detective agents:
    - repo_investigator
    - doc_analyst
    - vision_inspector

    These agents operate in complete isolation and parallelism, each executing its forensic protocol (AST parsing, PDF analysis, etc.) without interfering with the others. This fan-out ensures that all evidence streams are gathered concurrently, minimizing total execution time.
2. **The Synchronization Point (Fan-In)**: All three detective branches converge on the evidence_aggregator node. This node waits for all parallel processes to complete before executing. It acts as a critical fan-in point, collecting the structured Evidence objects from each detective and synthesizing them into a unified evidence within the AgentState. This synchronized state is then passed to the next layer, ensuring the judges have a complete and consistent fact base.
3. **The Judicial Layer (Fan-Out)**: With the evidence prepared, the graph fans out a second time from the evidence_aggregator to the three judge nodes:
    - prosecutor
    - defense
    - tech_lead

    The three judge nodes run in parallel, each applying its unique, conflicting philosophy to the exact same evidence set. Their outputs are appended to the opinions list using the operator.add reducer, which concatenates lists rather than overwriting—critical for preserving all three viewpoints.
4. **The Supreme Court (Fan-In)**: The final fan-in point is the chief_justice node, which collects all three judicial opinions and applies deterministic rules to synthesize a final verdict.


START
Detective Layer (Forensic Analysis)
Fan-Out: Parallel Investigation
RepoInvestigator\n(AST, Git Log)
DocAnalyst\n(RAG-lite on PDF)
VisionInspector\n(Image Analysis - Optional)
Parallel execution ensures forensic analysis is fast and evidence is gathered from multiple sources simultaneously.
Evidence (Code Facts)
Evidence (Report Facts)
Evidence (Diagram Facts)
Fan-In: EvidenceAggregator
Aggregated Evidence Map
Judicial Layer (Dialectical Debate)
For each Rubric Dimension
Dialectical Synthesis is achieved by forcing three conflicting personas to debate the same evidence for each criterion.
Fan-Out: The Bench
Prosecutor\n(Critical Lens)
Defense\n(Optimistic Lens)
Tech Lead\n(Pragmatic Lens)
JudicialOpinion (Score, Argument)
JudicialOpinion (Score, Argument)
JudicialOpinion (Score, Argument)
Fan-In: Collect Opinions
List[JudicialOpinion]
Supreme Court (Synthesis Engine)
Apply Hardcoded Rules:\n- Security Override\n- Fact Supremacy\n- Variance Check
Generate AuditReport
final_report.md
Metacognition: The system resolves conflict using deterministic rules, not an LLM, ensuring auditability.

Dialectical Synthesis: From Thesis, Antithesis, to Verdict

Dialectical Synthesis is the engine of our "Digital Courtroom." It is not achieved by averaging scores but by structuring a conflict between three distinct personas and then resolving it with a set of hardcoded, logical rules.

Thesis (Prosecutor): The prosecutor_node is programmed with a cynical, detail-oriented persona. Its prompt instructs it to look for missing pieces, security flaws, and violations of the rubric's letter. Its output is a harsh, critical opinion.

Antithesis (Defense): The defense_node is the empathetic counterpart. Its persona is forgiving and assumes good intent. It looks for effort, clever workarounds, and the "spirit of the law," often arguing for partial credit where the prosecutor sees only failure.

Synthesis (Chief Justice): The chief_justice_node does not use an LLM to form a compromise. Instead, it implements a deterministic synthesis engine. It ingests the conflicting opinions and applies hardcoded rules (e.g., "Security Override," "Fact Supremacy") to resolve the conflict. For example, if the prosecutor cites a confirmed security flaw (os.system with unsanitized input), the Chief Justice's rule engine will automatically overrule any effort-based points from the defense and cap the score. This process transforms raw conflict into a structured, justifiable verdict, explicitly citing the dissent that led to the final decision.

Metacognition: The System Thinking About Its Own Judgment

Metacognition in this system is the ability to not just produce an answer, but to audit its own reasoning process. This is achieved through two key mechanisms:

1. **Explicit Dissent Tracking**: The synthesis engine is required to identify and summarize disagreements. In chief_justice_node, the variance between judge scores is calculated. If the variance exceeds a threshold (>2), the node creates a dissent_summary. This summary is not just a score; it's a meta-commentary on the evaluation itself, explaining why the judges disagreed (e.g., "Prosecutor cites missing file X, Defense argues for effort based on complex AST logic."). This dissent is then included in the final report, allowing a human reader to understand the underlying tension in the evaluation.
2. **Structured, Traceable State**: The entire deliberation is captured in the AgentState. The evidences dictionary provides a traceable chain of facts, and the opinions list provides a traceable chain of reasoning. By using Pydantic models and reducers, we ensure that every step of the process is recorded in a structured, machine-readable format. The final AuditReport model then serializes this entire chain of thought, from raw evidence to final verdict, making the system's internal "thought process" transparent and auditable. This is the essence of building an evaluator that can evaluate itself.

3. Self-Audit Criterion Breakdown

The following breakdown presents the results of running the Automaton Auditor against my own repository. For each rubric dimension, the final score is presented alongside the three judge opinions and the evidence that informed them. This demonstrates the complete traceability from detective evidence through dialectical deliberation to final verdict.

Git Forensic Analysis

Final Score: 5/5

Judge	Score	Opinion Summary
Prosecutor	5	The repository contains 39 commits over an extended period with descriptive, atomic messages tracing logical development from environment setup through specification, architecture, feature implementation, and final refinements. Timestamps are spread across many hours, demonstrating incremental progress rather than bulk upload.
Defense	5	The commit history shows rich, incremental development well beyond minimal requirements. Starting with environment setup, successive commits add specifications, architecture, state reducers, and distinct features. A commit explicitly mentions wiring detectives in parallel within `src/graph.py`, evidencing graph orchestration.
Tech Lead	5	39 commits spanning from initial setup through specification, architecture definition, state reducers, feature additions, and crucial graph orchestration. Each commit has clear, descriptive messages with timestamps spread over days, indicating genuine step-by-step progress.

Evidence Cited: Commit hashes `b430a98c`, `04a820e0`, `6567fe33`, `ceaaf183`, `1a77ffe2`, `19930558`, `12af5150`, `888f06ce`, `aa943a60`, `99962b2f` showing progression from "init setup" through "feat: Judicial Layer" and "feat: Supreme court".

Synthesis Reasoning: All three judges unanimously agree on a perfect score. The evidence is irrefutable: 39 atomic commits with clear progression from environment setup to tool engineering to graph orchestration. No dissent required.

State Management Rigor

Final Score: 5/5

Judge	Score	Opinion Summary
Prosecutor	5	AgentState is a TypedDict that uses Annotated reducers (operator.ior for dict aggregation, operator.add for list concatenation) to safeguard parallel agent updates. Both Evidence and JudicialOpinion are Pydantic BaseModel classes with fully typed fields. This demonstrates correct, well-designed state handling.
Defense	5	The repository defines state management with rigorous typing and reducers. Evidence, JudicialOpinion, CriterionResult, and AuditReport are all Pydantic BaseModel classes with explicit typed fields. No plain dicts or missing reducers are observed.
Tech Lead	5	AgentState as TypedDict with annotated reducers prevents parallel agents from overwriting data. Both Evidence and JudicialOpinion are Pydantic BaseModel classes with typed fields. Fully satisfies the success pattern.

Evidence Cited: src/state.py containing AgentState(TypedDict) with evidences: Annotated[Dict[str, List[Evidence]], operator.ior] and opinions: Annotated[List[JudicialOpinion], operator.add].

Synthesis Reasoning: Unanimous agreement. The implementation perfectly matches the rubric's success pattern. The use of reducers is particularly critical for parallel execution safety.

Graph Orchestration Architecture

Final Score: 4/5

Judge	Score	Opinion Summary
Prosecutor	4	The graph defines two clear parallel fan-out/fan-in stages: Detective layer (repo_investigator, doc_analyst, vision_inspector) fans out from START and fans in at evidence_aggregator; Judicial layer (prosecutor, defense, tech_lead) fans out from aggregator and fans in at chief_justice. Missing conditional edges for error handling prevent a perfect score.
Defense	4	Two distinct parallel layers with proper fan-out/fan-in wiring. All required nodes are present and correctly connected. However, conditional edges for error states (fallback or retry branches) are absent.
Tech Lead	4	Parallel fan-out/fan-in for both Detectives and Judges matches required architecture. The only shortcoming is absence of explicit conditional edges for error handling.

Evidence Cited: src/graph.py showing explicit add_edge calls creating parallel branches:

1. builder.add_edge(START, "repo_investigator")
2. builder.add_edge(START, "doc_analyst")
3. builder.add_edge(START, "vision_inspector")
4. builder.add_edge("repo_investigator", "evidence_aggregator")

Synthesis Reasoning: All three judges concur on the score of 4. The core parallel architecture is fully implemented and correct. The missing conditional edges for error handling represent the only gap. The dissent requirement is not triggered as there is no variance.

Safe Tool Engineering

Final Score: 4/5

Judge	Score	Opinion Summary
Prosecutor	3	Uses subprocess.run with error handling and tempfile.TemporaryDirectory for sandboxing. However, no sanitization or validation of repository URL before passing to git command, and authentication failures not explicitly distinguished.
Defense	5	All Git operations executed within tempfile.TemporaryDirectory(). clone_repository uses subprocess.run with check=True and catches CalledProcessError. No raw os.system calls. Authentication failures reported via captured error messages.
Tech Lead	4	Adheres to success patterns: tempfile.TemporaryDirectory sandboxing, subprocess.run with error handling. Lacks explicit input sanitization of repo_url and doesn't differentiate authentication failures. Solid but missing minor safeguards.

Evidence Cited:

- src/tools/repo_tools.py: clone_repository function with subprocess.run([...], check=True) and try/except
- src/nodes/detectives.py: with tempfile.TemporaryDirectory() as tmpdir in repo_investigator_node

Synthesis Reasoning: Score variance exists (3 vs 5 vs 4) due to differing interpretations of the missing sanitization. The Chief Justice applies the Rule of Functionality: the Tech Lead's pragmatic assessment (4/5) becomes the final score, as it accurately balances the presence of core safety mechanisms against the absence of explicit sanitization.

Structured Output Enforcement

Final Score: 5/5

Judge	Score	Opinion Summary
Prosecutor	5	Judge implementation correctly enforces structured output by binding JudicialOpinion Pydantic model as a tool via .bind_tools() and invoking it through the chain. Prompt explicitly instructs LLM to use the tool. No plain prompt parsing present.
Defense	5	Implementation binds JudicialOpinion as a tool via llm.bind_tools([JudicialOpinion]), crafts prompt to require tool invocation, extracts tool call, coerces fields, and falls back to minimal opinion if missing. Guarantees schema validation.
Tech Lead	5	Judges enforce structured output by binding JudicialOpinion as a tool. Prompt explicitly instructs LLM to use only that tool. Response handling extracts tool calls, validates against schema before adding to state. No free-form parsing.

Evidence Cited: src/nodes/judges.py showing llm.bind_tools([JudicialOpinion]) and tool-call extraction logic.

Synthesis Reasoning: Unanimous agreement. The implementation exactly matches the success pattern: structured output enforcement with fallback handling and schema validation.

Judicial Nuance and Dialectics

Final Score: 5/5

Judge	Score	Opinion Summary
Prosecutor	5	Three separate node functions (prosecutor_node, defense_node, tech_lead_node) each with uniquely crafted persona prompts reflecting distinct philosophies: cynical prosecutor, empathetic defense attorney, pragmatic tech lead. Prompts differ substantially in tone and objectives.
Defense	5	Three clearly distinct judge personas, each with its own philosophy and prompt language. Node functions reference these personas separately, ensuring dialectical separation. Meets core requirement.
Tech Lead	5	Three separate node functions with distinct persona descriptions: Prosecutor cynical/detail-oriented, Defense empathetic/forgiving, Tech Lead pragmatic/tie-breaker. Prompts embed unique philosophical goals and wording, ensuring divergent reasoning.

Evidence Cited: src/nodes/judges.py showing three distinct prompt definitions:

- Prosecutor: "You are a cynical, detail-oriented prosecutor..."
- Defense: "You are an empathetic defense attorney..."
- Tech Lead: "You are a pragmatic Tech Lead..."

Synthesis Reasoning: Unanimous agreement. The implementation achieves genuine dialectical tension through carefully crafted, conflicting personas.

Chief Justice Synthesis Engine

Final Score: 5/5

Judge	Score	Opinion Summary
Prosecutor	5	chief_justice_node implements deterministic if/else logic with named rule branches (Tech Lead weight, Fact Supremacy, Security Override), includes dissent identification, and writes structured Markdown report with Executive Summary, Dimension Breakdown, and Remediation Plan.
Defense	5	Groups opinions, computes average base score, applies explicit rule branches for tech-lead weighting, fact supremacy penalties, security-override caps, detects dissent via score variance. Writes structured Markdown report matching success pattern.
Tech Lead	5	Fulfills deterministic rule-based synthesis requirements. Explicitly encodes security override, fact supremacy, architecture weight rules using clear if/else branches, applies variance-based dissent detection, writes structured Markdown audit report.

Evidence Cited: src/nodes/justice.py showing rule implementations:

Synthesis Reasoning: Unanimous agreement. The implementation perfectly matches the success pattern with no averaging-only fallback.

Theoretical Depth (Documentation)

Final Score: 2/5

Judge	Score	Opinion Summary
Prosecutor	1	Interim report provides superficial mentions of required terms. 'Dialectical Synthesis' and 'Metacognition' appear zero times; 'Fan-In' (2) and 'Fan-Out' (4) appear but without substantive architectural explanation tying them to graph edges or implementation.
Defense	2	Terms appear only a few times with no substantive architectural description. Counts show 'Dialectical Synthesis': 0, 'Metacognition': 0. Terms likely confined to high-level sections without implementation detail.
Tech Lead	2	Only token counts for key terms. Dialectical Synthesis and Metacognition absent. Fan-In/Fan-Out appear a handful of times but without linkage to specific graph edges or implementation details. Matches failure pattern.

Evidence Cited: reports/interim_report.pdf - term frequency analysis: {'Dialectical Synthesis': 0, 'Fan-In': 2, 'Fan-Out': 4, 'Metacognition': 0, 'State Synchronization': 0, 'Constitutional AI': 0, 'AST Parsing': 0}

Synthesis Reasoning: All judges agree on a low score. The documentation mentions required terms but fails to provide substantive architectural explanations connecting them to concrete implementation. This matches the failure pattern of "terms appear only in executive summary" rather than in detailed architectural explanations.

Report Accuracy (Cross-Reference)

Final Score: 4/5

Judge	Score	Opinion Summary
Prosecutor	5	Interim report lists six file paths. Cross-reference confirms each path exists in repository. No mismatches or hallucinated entries. Full compliance.
Defense	5	All six file paths present and verified. Feature claims accurately reflected in code base. Zero hallucinated paths.
Tech Lead	5	All six paths exist in repository. No missing or hallucinated paths. Feature claims align with actual code.

Evidence Cited:

- Claimed paths: src/nodes/detectives.py, doc_tools.py, judges.py, repo_tools.py, src/state.py, src/graph.py
- Verified: All six paths exist in the repository.

Synthesis Reasoning: Unanimous agreement on perfect accuracy. The report's file path claims are 100% verified against the actual repository.

Architectural Diagram Analysis

Final Score: 1/5

Judge	Score	Opinion Summary
Prosecutor	1	Interim report contains no images; PDF lacks any architectural diagram. Directly matches failure pattern of missing diagram. Critical failure.
Defense	1	No visual artifacts in PDF. No architectural diagram to assess, let alone one depicting parallel branches. Omission aligns with failure pattern.
Tech Lead	1	No diagrams in interim report. No visual representation of StateGraph, parallel branches, fan-out/in points, or flow matching architecture. Directly matches failure pattern.

Evidence Cited: reports/interim_report.pdf - "No images found in the PDF report."

Synthesis Reasoning: Unanimous agreement on critical failure. The complete absence of any diagram means the criterion cannot be satisfied. This represents the single largest gap in the submission.

MinMax Feedback Loop Reflection

The adversarial structure of this challenge—where our agent judges peers and their agents judge us—proved to be the most powerful mechanism for improvement. The shift from an initial self-audit score of 1.9/5.0 to the final 4.0/5.0 is a direct result of this MinMax optimization process.

.